图形接口与图形处理器的历史、发展现状和展望

贺俊哲
22421256

摘　要：本文回顾了图形接口（Graphics API）和图形处理器（GPU）的历史和发展现状，并探讨了其未来的挑战和发展方向。首先，本文介绍了主要图形接口的特性及应用，包括 OpenGL、DirectX、Vulkan 等。随后，本文分析了图形处理器的硬件架构和发展历程，从早期的二维图形处理器到现代高性能的多核 GPU。最后，本文展望了图形接口与图形处理器的未来展望，特别是以 WebGPU 为例的下一代图形接口，以及 GPU 在高性能计算、人工智能和实时渲染等领域的潜力与挑战。

关键词：图形接口（Graphics API），图形处理器（GPU），OpenGL， DirectX，Vulkan，实时渲染（Real-Time Rendering），光线追踪（Ray Tracing）

1 引言

在当今高度数字化的时代，图形接口和图形处理器已经成为计算机图形学以及相关领域中不可或缺的基础。随着游戏、虚拟现实、增强现实以及深度学习等技术的快速发展，对高效、强大图形处理能力的需求日益增长。图形接口（如 OpenGL、DirectX、Vulkan 等）和图形处理器（GPU）的进步为满足这一需求提供了关键支持。随着计算机图形技术的不断演进，图形接口和图形处理器在硬件和软件两个方面均经历了重大的变革和创新。

本文将深入探讨图形接口与图形处理器的发展历程，分析其现状，并展望未来的技术趋势。通过对主要图形接口的特性及其在不同场景中的应用进行介绍，以及对图形处理器的硬件架构和发展历史的详细解析，希望能够尽可能全面呈现图形接口与图形处理器的历史演变，并展望未来可能的发展方向。

本文的组织架构如下：

第 2 节概述了主要的图形接口，阐述了其基本功能，并介绍了目前主流的 OpenGL、DirectX、Vulkan 等。第 3 节详细介绍了图形处理器（GPU）的硬件架构和发展历程，从早期的二维图形处理器到现代多核高性能 GPU。第 4 节分别分析了当前图形接口和图形处理器的未来挑战和发展趋势，特别是以 WebGPU 为例的下一代图形接口，以及 GPU 在高性能计算、人工智能以及实时渲染领域的潜力与挑战。

2 主要图形接口

图形接口 (Graphics APIs) 作为计算机图形学中的关键组成部分，提供了一套标准化的应用程序接口 (API)，用于开发者创建和管理图形内容。一般来说，“API”是一组执行相关操作的标准函数集合，而“图形 API ”就是是一组执行基本操作（例如在屏幕上的窗口中绘制图像和 3D 表面）的函数。

计算机中每个图形程序都需要用到两个相关的 API：用于视觉输出的图形 API 、以及用于从用户获取输入的用户界面 API (GUI)。而本文所指的图形接口特指前者。

这些图形 API 均以某种形式的三维对象为输入，以显示在屏幕上的二维图片为输出，这个过程中进行的光栅化管道的基本计算都是相似的。在光栅化管道中，顶点通视图和投影矩阵从局部空间变换到全局空间、再转换为屏幕坐标。与几何体顶点对应的屏幕坐标被光栅化为片元（Fragment），最后变为像素；这些 API 也允许我们对这些片元设置纹理、照明和混合等操作。如图 1 所示。

目前主流的图形接口以 OpenGL 和 DirectX 为代表。而新一代图形接口则以 WebGL、Vulkan、 Metal 为代表。本节接下来就对这些图形接口的特性和应用进行简单的介绍。

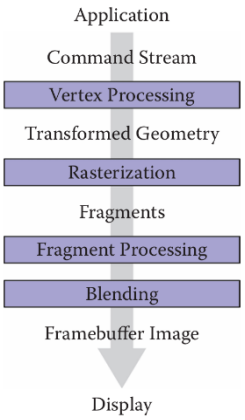


图 1. 基本图形硬件管道。[1]

[1] 图片来自[1]，第 17 章，p. 463。

2.1 OpenGL

OpenGL（Open Graphics Library）[2][3]是一个跨平台的图形 API，由 Silicon Graphics Inc.（SGI）在 1992 年推出。其目标是为不同操作系统和硬件平台提供统一的图形编程接口，以简化图形应用的开发过程。OpenGL 其后迅速成为了工业标准，广泛应用于 CAD、虚拟现实、科学可视化和游戏开发等领域。

OpenGL 的特性如下：

- **跨平台性**：支持多种操作系统，包括 Windows、macOS、Linux 等。更准确地说，OpenGL 是一个接口规范（Specification），规定了每个函数该如何执行，以及它们的输出值，因此它是独立于编程语言和平台的。
- **可扩展性**：当一个显卡公司提出一个新特性或者渲染上的大优化，通常会以扩展的方式在驱动中实现。如果一个程序在支持这个扩展的显卡上与运行，开发者可以使用这个扩展提供的一些更先进更有效的图形功能。通过这种方式，开发者不必等待一个新的 OpenGL 规范面世，就可以方便的检查显卡是否支持此扩展。
- 高效性：OpenGL 使用硬件加速的图形渲染技术，可以高效地处理大量的图形数据，实现高质量的图形渲染效果。
- 灵活性：OpenGL 提供了丰富的图形操作函数和工具，开发者可以自由地创建复杂的图形场景和动画效果。

不过，OpenGL 一般用于在图形工作站，PC 端使用，由于性能各方面原因，它难以应用到移动端，因而就有了下一节将介绍的 OpenGL ES。

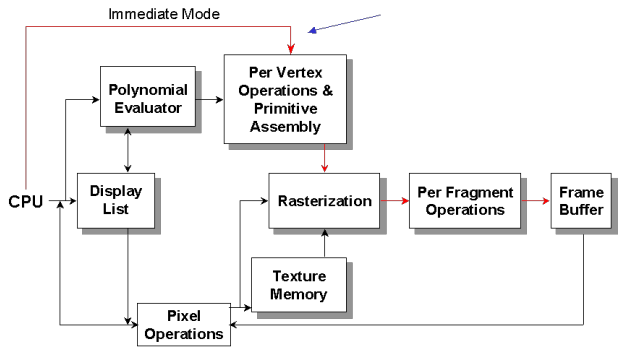


图 2. OpenGL 体系结构示意图。[2]

2.2 OpenGL ES

OpenGL ES（OpenGL for Embedded Systems）[4][5]是 OpenGL 的子集，是专门为嵌入式设备（如智能手机和平板电脑）设计的图形 API。其简化了 OpenGL 的功能，以适应低功耗设备的需求。在 Android 和 iOS 等移动端中得到了广泛应用。

具体而言，OpenGL ES (相比于 OpenGL) 的特点包括：

- 去除了 OpenGL 的冗余特性。在 OpenGL 中，执行相同操作的方式经常不止一种，而 OpenGL ES 中会只保留一种最有用的方法，并删除其它的冗余方法。例如，在 OpenGL 中有 3 种图形绘制方式：立即模式 (Immediate Mode)、显示列表 (Display List) 和顶点数组 (Vertex Array)，而 OpenGL ES 种仅保留了顶点数组。
- 尽可能保持与 OpenGL 的兼容。使用上一条中被保留了的 OpenGL 函数编写的程序往往也是有效的 OpenGL ES 程序。
- OpenGL ES 引入了新功能来解决移动设备和嵌入式设备的一些特定限制。比如，为了降低功耗和提高着色器的性能，在着色语言 GLSL 中引入了精度限定符。

目前为止，Khronos 已经发布了多个 OpenGL 规范，包括 OpenGL ES 1.X、OpenGL ES 2.X 以及 OpenGL ES 3.X，最新版本为 OpenGL ES 3.2[4]。OpenGL ES 1.X 提供了固定功能的加速支持；2.X 则提供了包括着色器 (shader) 在内的全可编程 3D 图形算法；3.X 提供了多个关于渲染目标、附加纹理功能、统一缓冲区、实例化和变换反馈的功能。

2.3 DirectX

DirectX [6]是由微软公司开发的一组多媒体编程接口，准确来说，其图形 API 部分应该被称作 Direct3D（D3D）[7]，而 DirectX 本身则包括了更多功能丰富的组件，包括 DirectDraw、DirectSound、DirectPlay、Direct3D、DirectInput、DirectSetup、AutoPlay 等。DirectX 广泛应用于 Windows 平台的游戏和多媒体应用开发。

[2]图片来自 https://csweb.wooster.edu/dbyrnes/cs300/Documents/CH2/ch2info.htm

DirectX 最早于 1995 年作为 Windows95 系统 API 的一部分推出，旨在在 Windows 平台中代替 OpenGL，提供硬件加速的多媒体编程接口。在 Windows 系统中，DirectX 和 图形设备接口 (GDI) 是处于同一地位的，如图 3 所示，只不过 GDI 一般用于 2D 绘图，主要用于桌面应用程序；DirectX 能够利用 GPU 的硬件加速，大幅提升绘图性能，更适合实时绘图或游戏开发应用。

DirectX 随着 Windows 版本的增加也在不断升级，每一次升级，都引入了新的功能，增强了 API 的性能和易用性。例如，DirectX 11 引入了计算着色器，DirectX 12 则更加注重底层硬件访问，显著降低了 CPU 开销，提高了并行处理能力。目前最新的版本为 DirectX 12 Ultimate，于 2020 年发布[8][9]。

Direct3D 作为 DirectX 中的图形 API，目前广为各家显卡所支持，与 OpenGL 同为电脑绘图软件和电脑游戏最常使用的两套绘图编程接口之一。

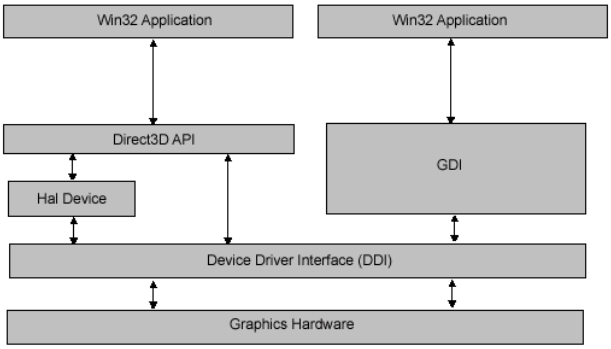


图 3. Window 应用程序、Direct3D、GDI 和硬件之间的关系。[3]

2.4 OpenCL

OpenCL（Open Computing Language）[10]是一个用于跨平台并行编程的框架，广泛应用于计算图形和通用计算任务，由 Khronos Group 推出，用于编写在 CPU、GPU 以及其他处理器上运行的并行计算程序。

相比于 OpenGL， OpenCL 更多地用于执行通用计算任务，如科学计算和数据处理，而非渲染图形。而且 OpenCL 支持在多种处理器上运行，如 CPU、GPU 和其他加速器，并不像 OpenGL 一样一般在 GPU 上允许。同时， OpenGL 主要关注顶点和片段处理，使用固定功能管线进行操作。 OpenCL 则专注于数据并行执行，可以支持更多自定义数据处理任务。由于其灵活性，OpenCL 也经常用于 AI、深度学习、医学成像等领域。

OpenCL 的计算结果可以直接用于 OpenGL 的图形渲染，实现计算与渲染的无缝衔接。这种结合应用的方式可以充分发挥两种技术的优势，提高整体的计算和渲染性能。在实际应用中，开发者也可以根据需求将两者结合使用：比如在游戏开发中，可以用 OpenGL 进行 3D 场景的渲染，同时使用 OpenCL 进行物理模拟、人工智能等复杂计算任务的处理；通过合理利用两种技术的优势，可以实现更加逼真的游戏效果和更高的计算性能。

2.5 WebGL

WebGL（Web Graphics Library）[11][12][13] 是基于 OpenGL ES 的网页图形 API。在 WebGL 出现之前，开发人员必须依赖插件或其它的应用程序，还得要求用户下载和安装某些软件才能提供硬件加速的 3D 图形绘制。而 WebGL 出现之后，开发人员可以使用 JavaScript 在浏览器中充分利用计算机 3D 渲染硬件的全部功能，其流程如图 4 所示。虽然 WebGL 不在官方 HTML5 规范中，但大多数支持 HTML5 的浏览器都提供了 WebGL[13]。

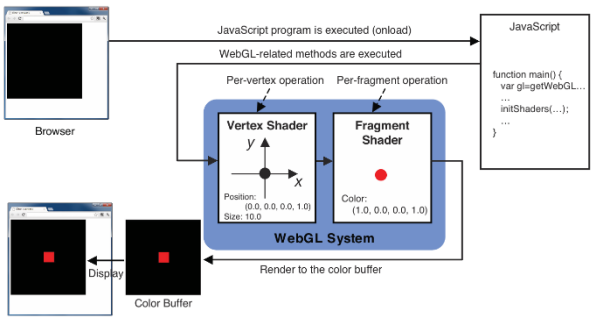


图 4. 从运行 JavaScript 到在浏览器中显示结果的处理流程。[4]

WebGL 的一些重要特性如下：

- 可通过 JavaScript API 直接调用，没有像 HTML 那样的附带标签。
- 基于 OpenGL ES 。WebGL 的设计者认为，将 API 基于占用空间更小的 OpenGL ES ，会更容易为网页提供一致、跨平台、跨浏览器的 3D API。最新的 WebGL 2.0 版本基于 OpenGL ES 3.0 API[11]，见图 5。
- 与其它网页内容结合。例如，用 WebGL 开发时，可以直接用画布 <canvas> 元素来绘制

[3] 图片来自 https://learn.microsoft.com/en-us/windows/win32/direct3d9/direct3d-architecture

[4] 图片来自[13]，p. 27。

WebGL 渲染的内容，其它的网页元素仍然可以用 HTML 来写。

- 跨平台。和 OpenGL ES 一样，WebGL 能够在任何操作系统上运行，并且适用于从手机、平板电脑到台式电脑的各种设备。

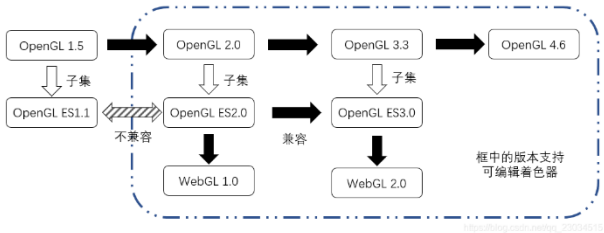


图 5. WebGL，OpenGL 和 OpenGL ES 三者的版本关系。[5]

2.6 Metal

Metal[15][16][17]是苹果公司于 2014 年推出的一种现代图形和计算 API，兼顾图形与计算功能，面向底层、低开销的硬件加速。Metal 专为 Apple 平台而设计，包括 iPhone、iPad、Mac、Apple TV 和 Apple Vision Pro。由于 Metal 是 Apple 平台独有的，因此它不能像 OpenGL ES 那样支持跨平台，但是它能最大地挖掘 Apple 设备的 GPU 能力，进行复杂的运算，Unity 等游戏引擎都通过 Metal 对 3D 能力进行了优化[18]， App Store 还有相应的运用 Metal 技术的游戏专题。

Metal 特点包括[19]：

- 低 CPU 开销；
- 最佳 GPU 性能；
- 持续保持处理器的并行性；
- 高效的资源管理。

同时，Apple 提供了很多可供开发者进行 GPU 级别的调试工具以及性能分析工具，例如 Metal Debugger、Metal System Trace 等，可以帮助比较庞大而复杂的工程突破性能瓶颈[20]，同时提供更加优秀的用户体验。

2.7 Vulkan

Vulkan [21][22]是 Khronos Group 于 2016 年推出的一个新一代图形 API，跨平台，旨在取代 OpenGL 及 OpenGL ES、提供更高效的图形性能和更低的 CPU 开销。Vulkan 更加轻量级，并有更贴近底层硬件(close-to-the-metal)的接口、有更细粒度的控制和并行计算能力，可使 GPU 驱动软件运用多核与多线程 CPU 性能。

Vulkan 的重要目标就是性能的提升，因而被设计为一种“底层”API。对于 OpenGL 这种传统的 API 而言，驱动程序完成了大部分处理工作，包括是否以正确的方式使用 API 调用，是否传递了正确的参数，以及状态是否准备充分，若出现问题，则提供反馈。但是 OpenGL 为了这些事情牺牲了大量的性能。因此，Vulkan 把 API 验证、内存管理、多线程管理等工作交由开发者负责，如图 6 所示。一旦 API 使用出错，应用就会宕机。这种方式无疑增加了 API 使用的复杂度和困难度，但换来的是性能上巨大的提升。

此外，在 OpenGL 中，所有的渲染操作都放在一个线程，限制了性能。Vulkan 中引入了 Command Buffer 的概念[23]，如图 7 所示；每个线程都可以往 Command Buffer 提交渲染命令，给开发者提供了充分发挥 CPU 多核多线程的优势。

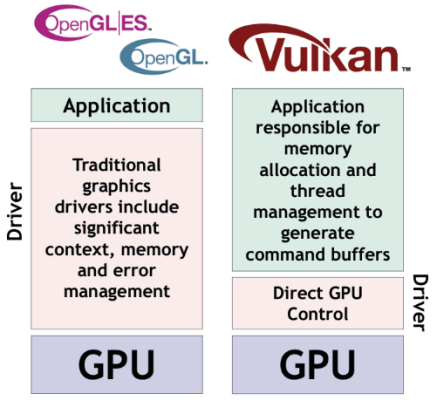
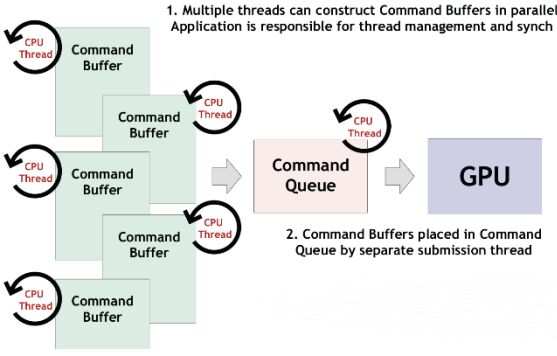


图 6. 相比于 OpenGL, Vulkan 实采用显式的 GPU 控制。[6]



[5] 图片来自 https://blog.csdn.net/qq_23034515/article/details/108283747

[6] 图片来自 https://www.khronos.org/assets/uploads/developers/library/overview/Vk_201602_Overview_Feb16.pdf

图 7. Vulkan 中的 Command Buffer 可以充分发挥 CPU 多核多线程的优势 [7]。

2.8 总结

我们将前文介绍的图形接口整理为表 1。

表 1. 图形接口总结

名称	发布时间	开发者/维护者	适用场景
OpenGL	1992	Khronos Group	计算机图形渲染、视频游戏开发以及工程应用程序。
OpenGL ES	2003	Khronos Group	专为移动和嵌入式系统（如智能手机、平板电脑、游戏机和车载系统）设计。
DirectX	1995	Microsoft	Windows 和 Xbox 上的游戏和多媒体应用。
OpenCL	2009	Khronos Group	异构平台上的并行编程。
WebGL	2011	Khronos Group	在 Web 浏览器中的 3D 图形渲染。
Metal	2014	Apple	iOS、iPadOS、macOS、tvOS 上的图形和计算。
Vulkan	2016	Khronos Group	跨平台的高性能图形和计算。

3 图形处理器

随着半导体工艺水平的不断提升和计算机体系结构的不断创新，图形处理器 (GPU) 在过去的二十多年间得到了快速发展 [8]。

3.1 GPU 硬件简述

图 1 已经展示了图形 API 的一般操作；在实际中，这些操作所带来的复杂运算均由 GPU 完成，如图 8 所示。可以总结如下：

- 用户操作的图形应用在 CPU 上运行；该应用将三维物体的顶点以及相关的颜色、纹理信息，所谓图元 (primitives) 信息；这些信息通过图形 API 发送到 GPU。
- 在 GPU 中，顶点着色器 (vertex shader) 会对三维对象进行一系列变换，包括几何变换，以及用视图矩阵将世界坐标转换到相机坐标、用投影矩阵将相机坐标转换到像素坐标。几何着色器 (geometry shader) 基于输入的图元生成新的图元、或对输入图元进行修改。
- GPU 将图元进行适当的裁剪，颜色和纹理数据也相应由像素着色器 (pixel shader) 作出必要的调整，并确定帧缓存 (frame buffer) 中的像素值。写入帧缓存的过程将图元转变为一个由像素组成的二维图象，称作光栅化 (rasterization)，由光栅化处理单元 (ROP) 完成。但这些二维图像同时包含颜色和纹理等信息，原来的图元此时称作片元 (fragments)。
- 最后 GPU 进行检测和确定输出。检测包含模板检测、深度检测 (丢弃被遮挡的像素) 、Alpha 检测与混合等，确定最终显示到屏幕中的像素。

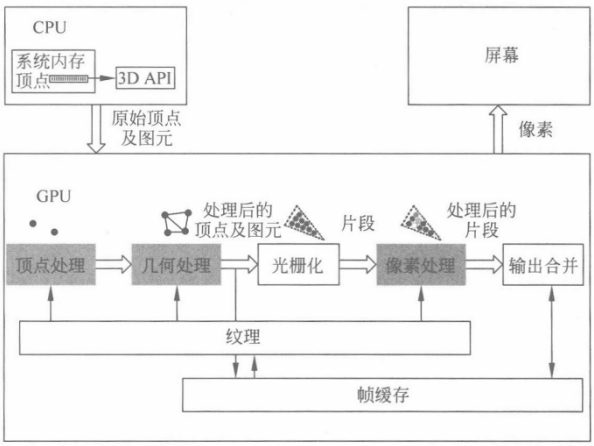


图 8. GPU 处理流水线 [9]，灰色代表可编程部分。

为了处理密集的计算，现代 GPU 由多核组成，如图 9 所示。这些核被 NVIDIA 称作流式多处理器 (streaming multiprocessors)，而 AMD 则称之为计算单元 (compute units)。每个 GPU 核都会运行一个单指令

[7] 图片来自 https://www.khronos.org/assets/uploads/developers/library/overview/Vk_201602_Overview_Feb16.pdf

[8] GPU 这一概念是在 1999 年由 Nvidia 首次提出。虽然本文沿用了这种说法，但实际上更早期的图形加速器也可以视作属于 GPU，见 3.2 节。

[9] 图片来自[22] 1.2.1 节，p. 5。

多线程 (SIMT) 程序，这里涉及到很多硬件和体系结构工程师需要考虑的问题，包括进程间通信、进程同步、数据缓存、缓存一致性问题，这里不再叙述。

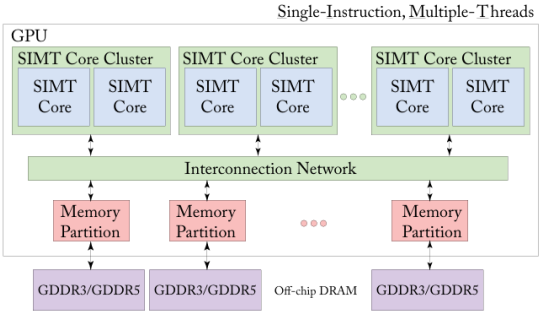


图 9. 现代 GPU 架构[10]。

3.2 历史和发展现状

3.2.1 早期阶段

人们早在上世纪六七十年代就已经发现：图形渲染对于计算能力有很强的需求，因此需要使用专用硬件来完成生成图像所需的计算，彼时这种硬件被称作寻址器或适配器，作为主处理器和显示器之间的通道。这段时期的设计创新为 GPU 奠定了重要基础。

早期的图形处理器主要用于专业和科学计算，例如在航天和军事领域，同时这些处理及基本都只能进行二维的位图操作，直到 90 年代末才出现了硬件加速的三维坐标转换和光源计算技术。随着计算机技术的进步，图形处理器逐渐演变成现代意义上的 GPU。

一些代表性的技术见表 2。

表 2. 20 世纪图形处理代表技术产品

名称	时间	简介
CDP1861 "Pixie" 视频芯片	1976	输出 62×128 分辨率的 NTSC 兼容的视频信号，或为 RCA Workshop II 控制台输出 64×32 分辨率的视频信号。
电视接口适配器 (TIA)	1977	负责生成屏幕显示、音效并读取输入控制器。
Motorola 6845 (MC6845)	1980s	定时访问显存，并计算下一个要绘制部分的内存地址。
IBM 颜色图形适配器 (CGA)	1981	内置 16 KB 显存，有多种图形和文本模式。最高显示分辨率为 640 × 200，支持的最高色深为 4 位。
字母数字电视接口控制器 (ANTIC)	1981	使用 DMA 读取显示指令来绘制每一帧图形。
NEC μPD7220 (Intel 82720)	1981	直接控制最大为 1024×1024、4 位色深的显存，可以进行地址计算、图形绘制、光栅扫描同步、DMA 控制等，无需其它硬件。[26]
IRIS 1000	1984	提供图形光栅显示功能。
ATI Graphics Solution	1985	通过 DIP 开关支持 MDA、CGA 和 EGA 显卡模式。
ATI VGA Wonder	1988	采用 VGA 接口，支持 1024×768 的分辨率与 24 位色深。
Mach8 (ATI38800)	1991	ATI 的首个图形加速器，可以绘制线段、矩形、多边形等图元，与 IBM 8514/A 兼容。
ATI Mach64	1994	64 位加速器，支持从 YUV 到 RGB 色彩转换、动态视频加速等。
GLINT 300SX	1994	支持 Gouraud 着色、深度缓冲、抗锯齿、Alpha 混合。
trio64v+ (S3 765)	1995	最多支持 2MB EDO 显存，支持高分辨率，在低分辨率下可以实现 32 位真彩色。
Nvidia NV1	1995	Nvidia 首款产品，基于二次曲面贴图的方式实现 3D 效果。
3Dfx Voo-Doo	1995	被认为是第一款个人消费级显卡。
ATI 3DRage	1996	ATI 的第一款 3D 显示芯片
Nvidia	1997	NVIDIA 第一款对微软

[10] 图片来自[23] 1.2 节，p. 4。

RIVA 128		Direct3D 加速提供支持的图形芯片，像素填充率达到100Mpixel/s，并首次实现硬件三角生成引擎。
Nvidia GeForce 256	1999	被认为是世界上第一款GPU[27][28]。首款集成了变换、光照、三角形构成、裁剪、纹理和染色引擎的芯片与OpenGL 1.2和DirectX7.0全兼容。

20 世纪末期，显卡市场呈现 3Dfx、Matrox、Nvidia、ATI 几家激烈竞争的局面，同时还有在 80 年代末入局的 S3 Graphics、Trident、SiS 等一众厂商也参与其中。至 21 世纪才最终得到了如今 Nvidia 和 AMD (ATI) 两分天下的局面。

3.2.2 21 世纪的 GPU

2000 年代初期，NVIDIA 和 ATI（后来被 AMD 收购）成为了 GPU 市场的两大主要竞争者，二者在这段时期同样推出了一系列创新产品，其中包括 NVIDIA 的 GeForce 和 ATI 的 Radeon 系列。

1999 年的 GeForce 256 首次将变换和光照的顶点计算与片段计算整合在了同一芯片上：这种方法将顶点计算从 CPU 转移出去可以提高游戏的几何复杂性，但代价是对浮点运算能力的显著需求[28]。随着 PC 游戏变得越来越复杂，修改顶点和像素计算以实现更复杂图形效果的需求也随之增长。2001 年，Nvidia 推出的 Geforce3 和以往的固定流水线不同，首次提出了可编程的顶点着色器，同时符合符合 DirectX8 API 的规定；2003 年的 GeForce 5(FX) 则进一步提出了可编程的片元着色器 (fragment shader)。

固定图形流水线需要用 CPU 计算出每一帧的顶点变化，传递给 GPU 流水线执行。但顶点着色器出现后，CPU 只需要把顶点数据准备好，然后在顶点着色器中编程控制顶点的各种属性就可以了。这样一来，CPU 只需要传递一次数据给 GPU，大大节省了数据传输消耗的时间。而且 GPU 强大的并行计算能力,使得在 GPU 中进行计算比在 CPU 中快得多。由于可编程性的引人，GPU 不再是一个功能单一的设备，拥有了更好的可扩展性和适应性[25]。

在 Geforce3 发布的同一年，ATI 发布了 Radeon 7500；进一步， 2002 年 10 月，ATI 推出了首个符合 DirectX 9.0 规范的加速器 Radeon 9000 系列，顶点和片元着色器可以像 CPU 一样灵活实现循环和长浮点数学运算。

[11] 图片来自［22］1.2.2，p.8。

2006 年，Nvidia GeForce8 (G80) 系列发布，拥有大量统一的可编程流处理器，也是首款支持 DirectX 10 的显示芯片。在此之前，GPU 一般采用固定比例的顶点着色器和片元着色器，但这会导致计算单元利用率低下；而 GeForce8 的统一着色器架构解决了这一问题。这段时期，GPU 开始从单纯的图形处理器向通用计算设备转变。

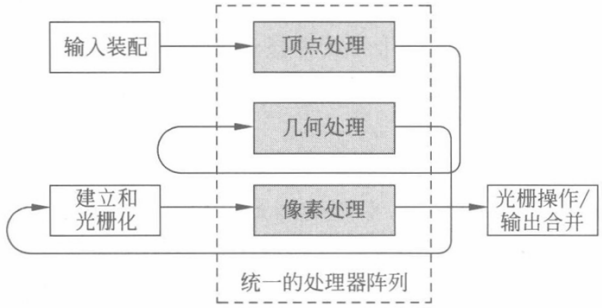


图 10. 逻辑图形流水线向统一的处理器阵列结构映射 [11]。

2007 年，CUDA（Compute Unified Device Architecture）技术的引入，大大降低了基于 GPU 开发通用应用程序的障碍，使得 GPU 在科学计算、数据分析等领域得到了广泛应用，极大地提升了计算效率。其他的 GPU 通用计算标准也被相继提出，例如 2.4 节介绍的 OpenCL。目前，CUDA 和 OpenCL 是两个最常见的 GPU 通用计算 API 标准。如表 3 所示。

表 3: OpenCL 和 CUDA 的对比。

API	发布者	支持平台	支持的GPU厂商	特点
OpenCL	Khronos Group	Windows、Linux、MacOS、Android等	国际：Nvidia、AMD、Intel等；国内：摩尔线程、景嘉微、芯动科技、格兰菲等	可以利用多种设备进行计算，包括CPU、GPU、DSP、FPGA等；支持异构计算；开发门槛较高。
CUDA	Nvidia	Windows、Linux、MacOS等	国际：Nvidia；国内：摩尔线程等	硬件与软件高度匹配；开发门槛较低。

GPU 的发展历程总结为表 4。

表 4 GPU 发展历程[29]。

时间	80年代	80 年代末	90 年代初	90 年代后期	2004~2010	2011~至今
类型	图形显示	2D 加速	部分 3D 加速	固定管线	统一渲染	通用计算
相关标准	CGA, VGA	GDI, DirectFB	OpenGL(1.1~4.1), DirectX(6.0~11)		CUDA, OpenCL1.2~2.0	

代表产品	IBM 5150	S3 86C911	Glint300SX	Ge-Force256	G80	TESLA
基本特征	光栅生成器	2D 图元加速	硬件 T&L	着色器功能固定	多功能着色器	完成与图形处理无关的科学计算

21 世纪初的 GPU 代表技术总结在表 5 中。

表 5. 21 世纪初图形处理代表技术产品

Nvidia	ATI / AMD	时间
Geforce 3	Radeon 7000	2001
Geforce 4	Radeon 8000/9000	2002
GeForce 5 (FX)		2003
Geforce 6/7	Radeon X800	2004
	Radeon X1000	2005
Geforce 8		2006
	Radeon HD 2000 / 3000	2007
GeForce GTX 280/260	Radeon HD 4000	2008
Geforce 300/400	Radeon HD 6000	2010
GeForce 600/700	Radeon HD 7000 Radeon R9 200	2012/2013
Geforce Gtx 10	Radeon RX400	2016
TITAN V	Radeon RX Vega	2017
Geforce Rtx 20	Radeon RX500	2018
GeForce Gtx16	Radeon RX 5000	2019
Geforce Rtx 30	Radeon RX 6000	2020
	Radeon RX7000	2022
Geforce Rtx 40	Radeon PRO W7000	2023

随着集成电路技术的发展，Nvidia 和 AMD 的 GPU 产品也在不断进行核心建构的更新。以 Nvidia 为例，从早期的 Tesla、Fermi 到最近的 Hopper、Lovelace、Blackwell，每一代都带来了显著的性能提升和技术创新，见图 11 和 [30]。

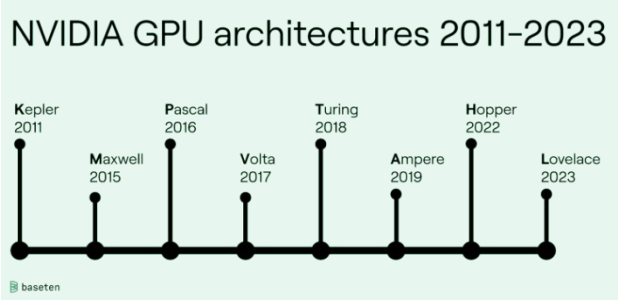


图 11. Nvidia GPU 核心架构变迁时间线 [12]。

4 未来趋势和展望

4.1 图形接口

4.1.1 现代图形接口的趋势

传统图形 API（DirectX11 及更早、OpenGL、OpenGL ES）的开销往往很大，主要表现在[17]：

- 状态校验（State validation）：
  - 确认 API 标记和数据合法。
  - 编码 API 状态到硬件状态。
- 着色器编译（Shader compilation）：
  - 运行时生成着色器机器码。
  - 状态和着色器之间的交互。
- 发送工作到 GPU：
  - 管理资源生命周期。
  - 批处理渲染命令。

而现代图形 API （Vulkan、D3D12、Metal 等）发布至今已有 7 年，随着行业的发展及主机世代的更迭，这些 API 也在持续不断地改进和提升。这些现代图形 API 有这些明显的特征：

- 图形+计算：不只是图形 API，而是 GPU 渲染 API 与计算 API 的结合。
- 底层：更低级的接口（更多的 GPU 直接控制），更少的（驱动）资源消耗，这种底层实现可以带来更少的运行时 API 验证，从而提高性能。这一性质在 Vulkan 上表现非常突出。
- 多线程：多个 CPU 线程可以并行调用图形

[12] 图片来自 https://www.baseten.co/blog/understanding-nvidias-datacenter-gpu-line/。

API 进行绘制，充分利用多核 CPU 性能。这也是 WebGPU 相对于 WebGL 的一个优势。

- 面向对象：相比于 OpenGL 的全局状态机，面向对象的编程模型不仅能够大大降低 IO 消耗，也更加方便编程实现。
- 跨平台兼容性：随着各种计算设备的多样化，图形 API 需要在多平台上提供一致的性能和效果。这样也可以简化开发者在不同平台之间移植应用的难度。

鉴于上述发展趋势，目前的现代图形 API 的设计也有趋同现象，如 Vulkan 和 Direct3D 12 中均出现了屏障 (Barrier)、描述子 (Descriptors) 的设计。

4.1.2 WebGPU：下一代 Web 图形 API

随着 Web 技术的飞速发展，图形渲染在 Web 应用中的重要性日益凸显。因此，我们可以把图形 API 按照如下方式分成两类：一种是 OpenGL、Metal、Vulkan、DX12 的普通图形 API；另外一种则是基于 WebGL、WebGPU 的 Web 图形 API。普通 API 更接近于底层硬件，可更好地发挥显卡能力；而 Web API 更多是利用浏览器环境。

由于 WebGL1/2 的设计来自 OpenGL ES，OpenGL ES 的普及率十分高，但是面对现代化的 CPU/GPU 设备和 Metal/Vulkan/D3D12 这些新生 API，性能和设计上显得较为陈旧。其局限性包括

- 缺乏对较新 GPU 功能的支持，例如计算着色器、光线追踪、VRS 可变着色率等。
- 由于需要验证每个 API 的调用并在 CPU 和 GPU 内存之间复制数据，因此 CPU 开销和内存使用率较高。
- 由于不同平台对于 OpenGL 的支持和优化存在差异，WebGL 在不同浏览器和平台之间的性能和表现也不一致。

因此 WebGPU [31][32]应运而生，可能会是最终取代 WebGL 的设计。WebGPU 的目标是为现代 GPU 提供更好的兼容、支持更通用的 GPU 计算、更快的操作以及能够访问到更高级的 GPU 特性。另一方面，其可能的应用也包括高性能网页游戏。与 WebGL 相比，WebGPU 在性能上有显著提升[33]。 2023 年，Google 宣布：从 Chromium/Chrome113 开始，Chromium/Chrome 浏览器将在支持 Vulkan 的 ChromeOS 设备、macOS 和支持 Direct3D 12 的 Windows 设备上提供 WebGPU 支持[31]。当下 Windows 平台的 Edge 浏览器和 Andorid 平台设备也已经开始支持了 WebGPU。后续，WebGPU 也将陆续支持像 Linux 和 IOS 等设备平台。

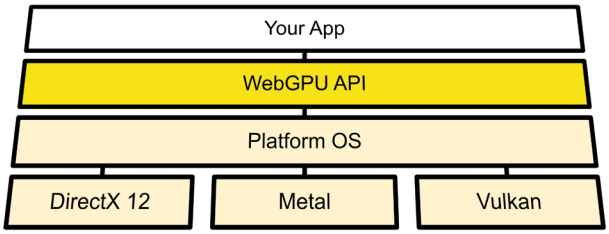


图 12. WebGPU 结构图 [13]。

随着 WebGPU 的成熟并获得广泛支持，我们可以期待各种各样的实际应用出现，包括[34]：

- 高性能网页游戏：创建视觉上更令人印象深刻、计算密集型的游戏，并且可以直接在浏览器中运行，而无需插件或额外的软件。
- 科学可视化和模拟：渲染复杂的 3D 可视化并运行模拟以进行科学研究和数据分析，从而利用现代 GPU 的处理能力。
- 机器学习和人工智能：执行机器学习任务并 (在浏览器中) 运行人工智能模型，从而实现一类新的智能 Web 应用程序。
- 虚拟和增强现实：支持基于网络的虚拟和增强现实体验，扩展网络上沉浸式内容的可能性。

不过，当今 WebGPU 还面临着开发环境的挑战，因为并非所有浏览器目前都支持 WebGPU，即使是那些支持 WebGPU 的浏览器也可能没有完全支持。

4.2 GPU

随着人工智能、深度学习和大数据分析的快速发展，GPU 在计算密集型任务中的应用越来越广泛。除了能效改进、性能提升等是无需强调的永恒的追求外，可以发现，GPU 发展趋势目前集中于[29]：

- 大规模扩展计算能力的高性能计算（GPGPU）；
- 人工智能计算（AIGPU）；
- 更加逼真的图形展现（光线追踪 Ray Tracing GPU)。

4.2.1 GPGPU

[13] 图片来自 https://developer.chrome.com/blog/new-in-chrome-94。

GPGPU [24][25]即“通用计算图形处理单元”，是一种将 GPU 的计算能力扩展到通用计算任务的技术。近十余年来，GPGPU 作为横跨图形/游戏,高性能计算、人工智能、云计算等多种行业应用的一种通用加速器件,得到了广泛的认同和快速的发展。得益于其 SIMT 计算架构，GPGPU 的并行计算能力出众，成为利用数据级并行 (Data-Level Parallelism) 提高任务处理性能的首选。配合不断完善的工具链和丰富的加速软件包，GPGPU 可以快速部署和应用到多种行业中,充分利用硬件能力取得明显的加速效果,成为各领域应用加速的典型范例。

GPGPU 正经历着快速的技术变革，并朝着领域更加多样、计算更加快速的方向迅速发展。其未来的挑战、同时也是重点关注的发展方向包括：

- 能耗问题：随着 GPGPU 计算能力的不断提升和存储器的持续升级，能耗问题也愈发突出，而某些领域如自动驾驶、智能设备，对能效有更高的要求。
- 扩展性及虚拟化的支持。受限于半导体的工艺集成能力和功耗，单芯片 GPGPU 计算能力仍然有限，大规模多芯片互联成为进一步提升算力的有效解决方案。在大规模部署的 GPGPU 系统架构中，如何在硬件、软件和系统层面支持虚拟化技术、更好地支持高性能计算和云计算等新型业务场景，是 GPGPU 支持各行各业发展的关键性问题。此外，某些应用单芯片 GPGPU 可能存在计算能力过剩的问题。通过虚拟化技术，让多个用户共享 GPGPU 芯片资源已经成为现实，但在效率和用户体验上有待进一步改善。
- 统一化软硬件接口：一方面，未来 GPGPU 的发展将扩展到 CPU 的计算领域。另一方面，包括 CPU 和 GPGPU 在内的多种硬件形态，如 ASIC 和 FPGA，仍然遵循各自的开发标准和使用习惯。未来，这些硬件形态的发展方向是对立还是统一？是否存在统一的计算描述方法和软件框架以适应不同硬件形态的转换？这是需要工业界和学术界共同思考和解答的问题。

4.2.2 人工智能计算

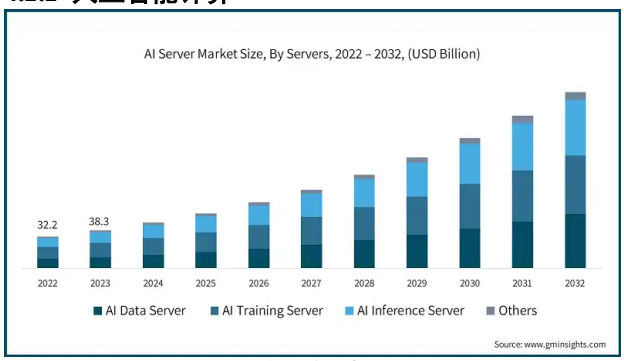


图 13. AI 服务器市场规模预测 [14]。

作为一种需要大量计算的领域，能够处理大规模并行计算任务的 GPU 也是人工智能和深度学习不可或缺的一部分。

GPU 的并行处理结构非常适合人工智能计算，但也需要专门为人工智能任务进行结构的调整。例如，但传统的基于流处理器的 GPU 一般只处理 FP32/FP64 等精度的运算，而 AI 计算的精度要求往往不高，INT4/INT8/FP16 往往可满足绝大部 分 AI 计算应用；因此，针对 AI 应用，Nvidia 设计了专用 的 Tensor Core 用于 AI 计算[35][36]，支持 INT4/INT8/FP16 等不同精度计算，显著加速深度学习训练和推断。

快速迭代的算法、不断增长的深度神经网络算力需求和复杂的神经网络模型结构演变对于 AIGPU 的可编程性、通用性和计算能力等方面都提出了更高的要求。如何在摩尔定律放缓的情况下支撑人工智能算法和算力的飞速发展，成为摆在 AIGPU 架构和电路设计者面前的难题。

4.2.3 光线追踪

光线追踪[37]的工作原理是模拟光线与虚拟场景中的物体和表面相互作用时的路径。与传统的光栅化技术（对每个像素逐一绘制）相比，光线追踪可以精确模拟反射、折射和阴影等复杂现象，让渲染结果更加有真实感。然而，光线追踪需要大量计算，尤其是在动态的场景中提供光线追踪，必须有强大的 GPU 支持。

Nvidia 的 Turing 架构 CPU 首次引入了专门为光线追踪（Ray Tracing）技术而设计的专用内核：RT Core，显著提升了光线追踪的效率，使得实时光线追踪变为可能。其采用的优化策略包括：

[14] 图片来自 https://www.gminsights.com/industry-analysis/ai-server-market。

- 混合渲染策略[38]：将传统光栅化技术与光线追踪相结合。在大多数场景中使用光栅化渲染，只有在需要精确光影效果的局部区域或特殊情况使用光线追踪。
- 算法优化：采用层级加速结构（如 BVH）来减少计算需求，并不断优化光线追踪算法，提升整体性能。
- AI 辅助优化：利用 Tensor Core 和深度学习算法进行去噪处理和 DLSS 技术，提高图像质量和渲染效率[39][40]。

类似地，AMD RDNA2 引入了称作“光线加速器(Ray Accelerator)”的光线追踪单元，实现了更充分的硬件资源利用。针对启用光线追踪功能后的性能优化，AMD 采用了可变速率着色（VRS）技术，通过动态降低屏幕中不需要高视觉细节的区域的遮光率，在画面细节几乎不受影响的前提下提升整体性能。

不过，实时光线追踪仍有很多问题需要攻克，如处理处理透明渲染、局部覆盖 (Partial Coverage)、去焦效果的实现，对多样的游戏内容环境进行更好兼容、全局光照[41]，以及波动光学渲染、体积渲染等复杂情况。同时，目前的技术距离真正的“实时”仍有差距，大量光线路径计算存在性能瓶颈；即使现代 GPU 中出现了硬件加速光线追踪功能，在实时应用中每个像素也只能追踪很少量的光线[42]，这需要在算法、GPU 的专用光线追踪元件、GPU 架构优化等方面共同取得更多突破。

5 总结

近些年，国外在图形接口与 GPU 技术快速发展，已经取得了较大成果，且不断在突破传统功能的范畴。同时，国内也在加快这些方面的技术进步，例如，国产图形引擎 Cocos、Oasis Engine、Ploy3D、Orillusion 都是利用现代图形 API 的优秀引擎。国内自研 GPU 也有诸如景嘉微、壁仞科技、芯动科技等领军企业。

但是，在图形引擎方面，国际上的渲染引擎如 Unreal Engine、Unity 等已经占据了市场主导地位国产渲染引擎，国内在技术和生态方面均存在较大差距。在 GPU 方面，国内 GPU 芯片的研制虽然可满足目前大多数图形应用需求，但在科学计算、人工智能及新型的图形渲染技术方面仍然和国外领先水平存在较大差距，未来持续发展国产 GPU 势在必行。如何研究和设计国产 GPU 是一个长久的话题。芯片供应链问题、国际大厂的竞争压力、技术方面的支持、架构的创新、生态的构建，都是国产 GPU 的严峻挑战。我国图形软硬件的发展事业仍然道阻且长。

参考文献

[1] Shirley, P., Ashikhmin, M., & Marschner, S. “Fundamentals of computer graphics (5th edition)”. AK Peters/CRC Press. 2022..

[2] Mark Segal, Kurt Akeley. "The OpenGL Graphics System: A Specification." Silicon Graphics Inc., 1992.

[3] Woo, M., Neider, J., Davis, T., & Shreiner, D. “OpenGL programming guide: the official guide to learning OpenGL, version 4.3”. Addison-Wesley Longman Publishing Co., Inc. 2013

[4] Khronos Group. .“OpenGL ES Overview”. https://www.khronos.org/opengles/.

[5] Ginsburg, Dan, et al. “OpenGL ES 3.0 programming guide”. Addison-Wesley Professional, 2014.

[6] Microsoft. “DirectX graphics and gaming”. Microsoft Learn. 2022. https://learn.microsoft.com/en-us/windows/win32/directx.

[7] Blythe, D. “The Direct3D 10 System,” ACM Trans. Graphics, Vol. 25, no. 3 (July), 724–34. 2006.

[8] Microsoft. "Direct3D 12 programming guide - Win32 apps." Microsoft Learn. 2021. https://learn.microsoft.com/en-us/windows/win32/direct3d12/directx-12-programming-guide.

[9] Shawn Hargreaves. “Announcing DirectX 12 Ultimate”. 2020. https://devblogs.microsoft.com/directx/announcing-directx-12-ultimate/.

[10] Gaster, Benedict, et al. “Heterogeneous computing with OpenCL”. Newnes, 2012.

[11] Khronos Group. .“WebGL - Low-Level 3D Graphics API Based on OpenGL ES”. https://www.khronos.org/webgl/.

[12] Parisi, Tony. “WebGL: up and running.” O'Reilly Media, Inc., 2012.

[13] Parisi, Tony. “Programming 3D Applications with HTML5 and WebGL: 3D Animation and Visualization for Web Pages.” O'Reilly Media, Inc., 2014.

[14] Matsuda, Kouichi, and Rodger Lea. “WebGL programming guide: interactive 3D graphics programming with WebGL”. Addison-Wesley, 2013.

[15] Apple. "Metal Overview." Apple Developer. 2021. https://developer.apple.com/metal/.

[16] Apple. "Metal | Apple Developer Documentation." Apple Developer, 2021. https://developer.apple.com/documentation/metal

[17] Apple. “Working with Metal: Overview - WWDC14 - Videos - Apple Developer”. Working with Metal: Overview - https://developer.apple.com/videos/play/wwdc2014/603/WWDC14 - Videos - Apple Developer

[18] Unity. “Unity - Manual: Optimize Metal graphics”. 2023. https://docs.unity.cn/Manual/metal-optimize.html

[19] Apple. “Metal Best Practices Guide: Fundamental Concepts.” (2017). https://developer.apple.com/library/archive/documentation/3DDrawing/Conceptual/MTLBestPracticesGuide/index.html

[20] Apple. “Metal Developer Tools - Apple Developer.” https://developer.apple.com/metal/tools/

[21] Vulkan. “Home | Vulkan | Cross platform 3D Graphics.” 2024. https://www.vulkan.org/

[22] Sellers, Graham, and John Kessenich. “Vulkan programming guide: The official guide to learning vulkan”. Addison-Wesley Professional, 2016.

[23] Vulkan. “Command Buffers : Vulkan Documentation Project.” https://docs.vulkan.org/spec/latest/chapters/cmdbuffers.html

[24] 景乃锋, 柯晶, 梁晓晓. “通用图形处理器设计: GPGPU 编程模型与架构原理”. 清华大学出版社. 2022.

[25] Aamodt, Tor M., et al. “General-purpose graphics processor

architectures.” Morgan & Claypool Publishers, 2018.
[26] Oguchi, T., et al. "A single-chip graphic display controller." 1981 IEEE International Solid-State Circuits Conference. Digest of Technical Papers. Vol. 24. IEEE, 1981.
[27] Peddie, Jon. “The History of the GPU-Eras and Environment.” Springer, 2022.
[28] Dally, William J., Stephen W. Keckler, and David B. Kirk. "Evolution of the graphics processing unit (GPU)." IEEE Micro 41.6: 42-51. 2021.
[29] 熊庭刚. "GPU 的发展历程, 未来趋势及研制实践." 微纳电子与智能制造 2.2: 36-40. 2020.
[30] NVIDIA. “NVIDIA Technologies & GPU Architectures for enterprise, developers, gaming, and industries”. 2024 . https://www.nvidia.com/en-us/technologies/.
[31] François Beaufort and Corentin Wallez. “Chrome ships WebGPU”. Chrome for Developers. 2023. https://developer.chrome.com/blog/webgpu-release.
[32] “WebGPU API”. MDN Web Docs. 2024. https://developer.mozilla.org/zh-CN/docs/Web/API/WebGPU_API.
[33] Usta, Z. “Webgpu: A new graphic api for 3d webgis applications”. International Society for Photogrammetry and Remote Sensing. 2024
[34] Baghai, C. “WebGPU: The future of graphics and compute on the web”. Medium. https://blog.devgenius.io/webgpu-the-future-of-graphics-and-compute-on-the-web-19f58ca23a12.
[35] Markidis, Stefano, et al. "Nvidia tensor core programmability, performance & precision." 2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW). IEEE, 2018.
[36] Choquette, Jack, et al. "Nvidia a100 tensor core gpu: Performance and innovation." IEEE Micro 41.2: 29-35. 2021.
[37] Appel, Arthur. "Some techniques for shading machine renderings of solids." Proceedings of the April 30--May 2, 1968, spring joint computer conference. 1968.
[38] Barré-Brisebois, Colin, et al. "Hybrid rendering for real-time ray tracing." Ray Tracing Gems: High-Quality and Real-Time Rendering with DXR and Other APIs: 437-473. 2019.
[39] Nvidia. “NVIDIA Deep Learning Super Sampling (DLSS)”. NVIDIA Developer. https://developer.nvidia.com/rtx/dlss.
[40] Watson, Alexander. "Deep learning techniques for super-resolution in video games." arXiv preprint arXiv:2012.09810. 2020.
[41] Colin Barré-Brisebois. “Are We Done With Ray Tracing? State-of-the-Art and Challenges in Game Ray Tracing”. 2019. https://media.contentapi.ea.com/content/dam/ea/seed/presentations/barrebrisebois2019-state-of-the-art-and-challenges-in-game-ray-tracing-release.pdf.
[42] Ouyang, Yaobin, et al. "ReSTIR GI: Path resampling for real‐time path tracing." Computer Graphics Forum. Vol. 40. No. 8. 2021.